

# MIP\_DOMAIN\_MODEL.md

## Mainframe Intelligence Platform

### Enterprise Domain Model

Version: 1.0

---

## Purpose

The Domain Model defines the canonical entities, attributes, relationships, and standards used across the Mainframe Intelligence Platform (MIP).

This model serves as the contract between:

- Discovery Layer
  - Metadata Repository
  - Knowledge Graph
  - APIs
  - Reasoning Layer
  - AI Copilot
  - Modernization Services
- 

## Domain Design Principles

### Principle 1

Every artifact becomes an entity.

Examples:

- Program
  - Job
  - Table
  - Copybook
  - Dataset
- 

### Principle 2

Relationships are first-class citizens.

Relationships often provide more value than artifacts.

Example:

PAYJOB → EXECUTES → PAYPOST

is more valuable than PAYJOB alone.

---

## Principle 3

Business context must be represented.

The goal is not code discovery.

The goal is business understanding.

---

## Core Domain Entities

---

### Program

Represents executable application logic.

Examples:

- COBOL Program
- PL/I Program
- Java Program

Attributes:

| Attribute    | Type      |
|--------------|-----------|
| id           | UUID      |
| name         | String    |
| language     | Enum      |
| program_type | Enum      |
| source_path  | String    |
| description  | String    |
| created_at   | Timestamp |
| updated_at   | Timestamp |

Program Types:

- Batch
- Online
- Utility
- Service
- Report

---

## Job

Represents batch execution units.

Attributes:

| Attribute   | Type   |
|-------------|--------|
| id          | UUID   |
| name        | String |
| scheduler   | String |
| frequency   | String |
| owner       | String |
| description | String |

Examples:

- PAYMENTJOB
- STATEMENTJOB

---

## Copybook

Represents shared structures.

Attributes:

| Attribute   | Type   |
|-------------|--------|
| id          | UUID   |
| name        | String |
| source_path | String |
| description | String |

---

# Table

Represents relational data structures.

Attributes:

| Attribute     | Type   |
|---------------|--------|
| id            | UUID   |
| name          | String |
| schema_name   | String |
| database_name | String |
| description   | String |

---

# Dataset

Represents file-based storage.

Attributes:

| Attribute    | Type   |
|--------------|--------|
| id           | UUID   |
| dataset_name | String |
| dataset_type | String |
| organization | String |

Examples:

- VSAM
  - Sequential
  - GDG
- 

# Transaction

Represents online transaction entry points.

Attributes:

| Attribute        | Type   |
|------------------|--------|
| id               | UUID   |
| transaction_code | String |
| description      | String |

Examples:

- CICS Transaction
- IMS Transaction

---

## Business Capability

Represents business functions.

Attributes:

| Attribute   | Type   |
|-------------|--------|
| id          | UUID   |
| name        | String |
| domain      | String |
| description | String |

Examples:

- Authorization
- Payment Processing
- Rewards
- Fraud Detection
- Statement Generation

---

## Business Rule

Represents business logic.

Attributes:

| Attribute | Type   |
|-----------|--------|
| id        | UUID   |
| rule_name | String |

| Attribute        | Type    |
|------------------|---------|
| rule_type        | String  |
| description      | String  |
| confidence_score | Decimal |

Rule Types:

- Validation
- Calculation
- Eligibility
- Workflow
- Compliance

---

## API Candidate

Represents future modernization opportunities.

Attributes:

| Attribute        | Type    |
|------------------|---------|
| id               | UUID    |
| name             | String  |
| capability       | String  |
| confidence_score | Decimal |

---

## Service Candidate

Represents potential microservices.

Attributes:

| Attribute        | Type    |
|------------------|---------|
| id               | UUID    |
| name             | String  |
| domain           | String  |
| confidence_score | Decimal |

---

# Repository

Represents application boundaries.

Attributes:

| Attribute       | Type   |
|-----------------|--------|
| id              | UUID   |
| name            | String |
| repository_type | String |

---

## Metadata Relationships

---

### EXECUTES

Source:

Job

Target:

Program

Cardinality:

One-to-Many

Example:

PAYMENTJOB EXECUTES PAYPOST

---

### CALLS

Source:

Program

Target:

Program

Cardinality:

Many-to-Many

Example:

PAYPOST CALLS PAYUPD

---

## USES

Source:

Program

Target:

Copybook

Cardinality:

Many-to-Many

Example:

PAYPOST USES CARDREC

---

## READS

Source:

Program

Target:

Table / Dataset

Cardinality:

Many-to-Many

---

## WRITES

Source:



Program

Target:

Dataset

Cardinality:

Many-to-Many

---

## UPDATES

Source:

Program

Target:

Table

Cardinality:

Many-to-Many

---

## IMPLEMENTS

Source:

Program

Target:

Business Rule

Cardinality:

Many-to-Many

---

## BELONGS\_TO

Source:

Program

Target:

Business Capability

Cardinality:

Many-to-One

---

## CONTAINS

Source:

Business Capability

Target:

Program

Cardinality:

One-to-Many

---

## PRODUCES

Source:

Job

Target:

Dataset

Cardinality:

Many-to-Many

---

## CONSUMES

Source:

Job

Target:

Dataset

Cardinality:

Many-to-Many

---

## Graph Node Standards

Every node must contain:

Required Attributes:

- id
- name
- type
- source\_system
- created\_at
- updated\_at

Optional Attributes:

- description
  - confidence\_score
  - tags
- 

## Graph Edge Standards

Every edge must contain:

Required:

- relationship\_type
- source\_id
- target\_id

Optional:

- confidence\_score
  - discovery\_source
  - created\_at
- 

## Confidence Model

Every discovered relationship should include confidence.

Example:

Program CALLS Program

Confidence:

0.99

Program IMPLEMENTS Business Rule

Confidence:

0.75

---

## Knowledge Graph Standards

Supported Traversals

Examples:

Job → EXECUTES → Program

Program → CALLS → Program

Program → UPDATES → Table

Program → USES → Copybook

Business Capability → CONTAINS → Program

---

## Enterprise Questions Supported

### Discovery

What programs exist?

What jobs exist?

---

### Dependency Analysis

Who calls this program?

Which jobs execute this program?

---

## Data Lineage

Which programs update this table?

Which jobs consume this dataset?

---

## Impact Analysis

What breaks if this copybook changes?

What breaks if this table changes?

---

## Business Understanding

What business capability owns this program?

Which rules are implemented?

---

## Modernization

What service candidates exist?

What APIs can be extracted?

---

## Future Domain Extensions

Planned Entities:

- MQ Queue
  - Kafka Topic
  - REST API
  - Event
  - Batch Chain
  - Cloud Service
  - Kubernetes Workload
  - Java Service
  - Spring Boot Application
-

# Versioning Strategy

Every domain model change must:

- Be backward compatible
  - Include migration scripts
  - Include graph migration strategy
  - Include API contract updates
- 

## Definition of Done

The domain model is considered complete when:

1. Every source artifact maps to a domain entity.
2. Every dependency maps to a relationship.
3. Every business concept maps to a business entity.
4. Every modernization insight maps to a modernization entity.

This domain model becomes the canonical language of MIP and the foundation for all future discovery, reasoning, and modernization capabilities.